DocRenew

Employee Document Renewal Management System

TIME ESTIMATION DOCUMENT

Time Estimation Document

DocRenew — Employee Document Renewal Management System

Prepared for	Zamorins Solutions
Prepared by	Anil B, Angular Developer — Zamorins Solutions
Contact	contact@zamorinstech.com
Document status	Draft v1.0
Date	14 October 2026
Basis	Entire build — coding, documentation, and testing — done using Claude

1. Purpose & Methodology

This document estimates the effort required to build DocRenew end to end — discovery through post-launch, including the Warranty, Vehicle Maintenance, Bill, AMC, and Other Notification categories added to the original employee-document scope — and projects a calendar timeline from that effort. It is the narrative companion to the task-level breakdown in DocRenew_Jira_Task_Sheet.xlsx, which lists all 73 tasks individually with the same hour estimates and is formatted for direct import into Jira.

The project is being built entirely by Anil B using Claude — for code generation, API design, UI implementation, documentation, and test support — rather than a multi-person team working unassisted or a developer writing code by hand. Estimates below reflect that:

- Coding, UI implementation, and documentation tasks are cut roughly 65–75% versus a fully manual, unassisted build, since Claude generates code, screens, and written deliverables directly and iterates on them in minutes rather than hours.
- Tasks bound by real people or real-world waiting — stakeholder meetings, legal/compliance review, cloud provisioning, user acceptance testing with live users, app store review, third-party onboarding — are reduced only modestly, because AI assistance does not compress those.

These are planning-level estimates, not a committed schedule. They should be revisited once the tech stack is finalized (Task T13) and after the first sprint of actual backend development, when real velocity data becomes available.

2. Total Effort Summary

Metric	Value
Total estimated effort	255 hours
Total tasks	73 tasks across 13 phases
Build approach	Solo developer, entire build done using Claude
Estimate basis	Planning-level, pre-sprint-1 — to be recalibrated after first sprint

3. Phase-by-Phase Breakdown

Hours per phase, with each phase's share of total effort:

Phase	Tasks	Hours	% of total
Discovery & Planning	5	10	4%
Requirements & UX Design	8	19	7%
Architecture & Tech Setup	6	18	7%
Backend Development	10	45	18%
Notifications & Integrations	4	16	6%

Phase	Tasks	Hours	% of total
Web Frontend Development	8	29	11%
Mobile App Development	5	22	9%
QA & Testing	7	33	13%
Security & Compliance	4	18	7%
Deployment & DevOps	4	12	5%
Documentation & Training	5	7	3%
Launch & Go-to-Market	4	16	6%
Post-Launch	3	10	4%

Total: 73 tasks, 255 hours.

Backend Development (45h), QA & Testing (33h), and Web Frontend Development (29h) are the three largest blocks, together accounting for just over 40% of total effort — consistent with this being a data- and workflow-heavy platform (the reminder/escalation engine, role-scoped APIs, and the compliance dashboard) rather than a simple CRUD app. The jump from the prior estimate reflects the added Warranty, Vehicle Maintenance, Bill, AMC, and Other Notification categories (tasks T67–T73), which extend the same data model and reminder engine to non-employee-owned items.

4. Calendar Timeline Projection

255 hours of effort translates to different calendar durations depending on how many hours per week go into the project. As a solo build, phases cannot run fully in parallel the way a multi-person team's would — some overlap is possible (e.g. starting frontend screens while backend endpoints for other modules are still in progress), but testing, review, and deployment remain serial bottlenecks.

Working pace	Hours / week	Approx. calendar duration
Part-time (evenings/weekends)	15 hrs/wk	~17 weeks (~4 months)
Dedicated part-time	25 hrs/wk	~10 weeks (~2.5 months)
Full-time	35–40 hrs/wk	~7 weeks (~1.5–2 months)

The full-time pace assumes no major scope changes and treats Mobile App Development (T37–T41, 22h) as in-scope for this release. If mobile is deferred to a Phase 2 release instead, total effort drops to roughly 233 hours, shortening the full-time estimate to about 6 weeks.

5. Assumptions & Risks

Assumptions

- One developer (Anil B) is the sole builder, with Claude used to generate code, UI, and documentation across the entire build rather than only assisting parts of it.
- Tech stack, cloud provider, and third-party integrations (email, SMS gateway, push notification service) are finalized early (Phase 3) and do not change mid-build.
- Stakeholder review and approval cycles (BRD/FSD sign-off, UAT feedback) do not introduce extended waiting periods beyond what's built into each task's estimate.
- Pilot customer / UAT participants are available when Phase 8 (QA & Testing) and Phase 12 (Launch) require them.

Risks to the estimate

- Third-party integration friction (SMS gateway approval, push notification service setup, email deliverability) is a common source of delay not fully within the developer's control — Notifications & Integrations (23h) could run over if any provider has a slow onboarding or verification process.
- App Store / Play Store review (within T41) has a review queue outside the project's control, typically 1–7 days per submission and sometimes longer on rejection and resubmission.
- Compliance and legal review (T50–T51) depends on availability of whoever signs off on data-privacy matters at Zamorins — if that's a single person with other priorities, this phase's calendar time can extend well beyond its 15 hours of actual work.
- As a solo build, there is no second developer to absorb illness, other project demands (EmailZapp, ParkZapp, LeadZapp), or context-switching cost — the calendar estimates above assume focused, uninterrupted time at the stated weekly pace.

6. Reference

The full task-by-task estimate — all 73 tasks with individual hour estimates, owners, priority, and status — is maintained in DocRenew_Jira_Task_Sheet.xlsx in this same folder, formatted for direct CSV import into Jira (Epic Name / Epic Link columns map to the 13 phases above).